



University  
of Glasgow

## W5-05: HDFS Reads and Writes

COMPSCI4064 (H) and COMPSCI5088 (M)

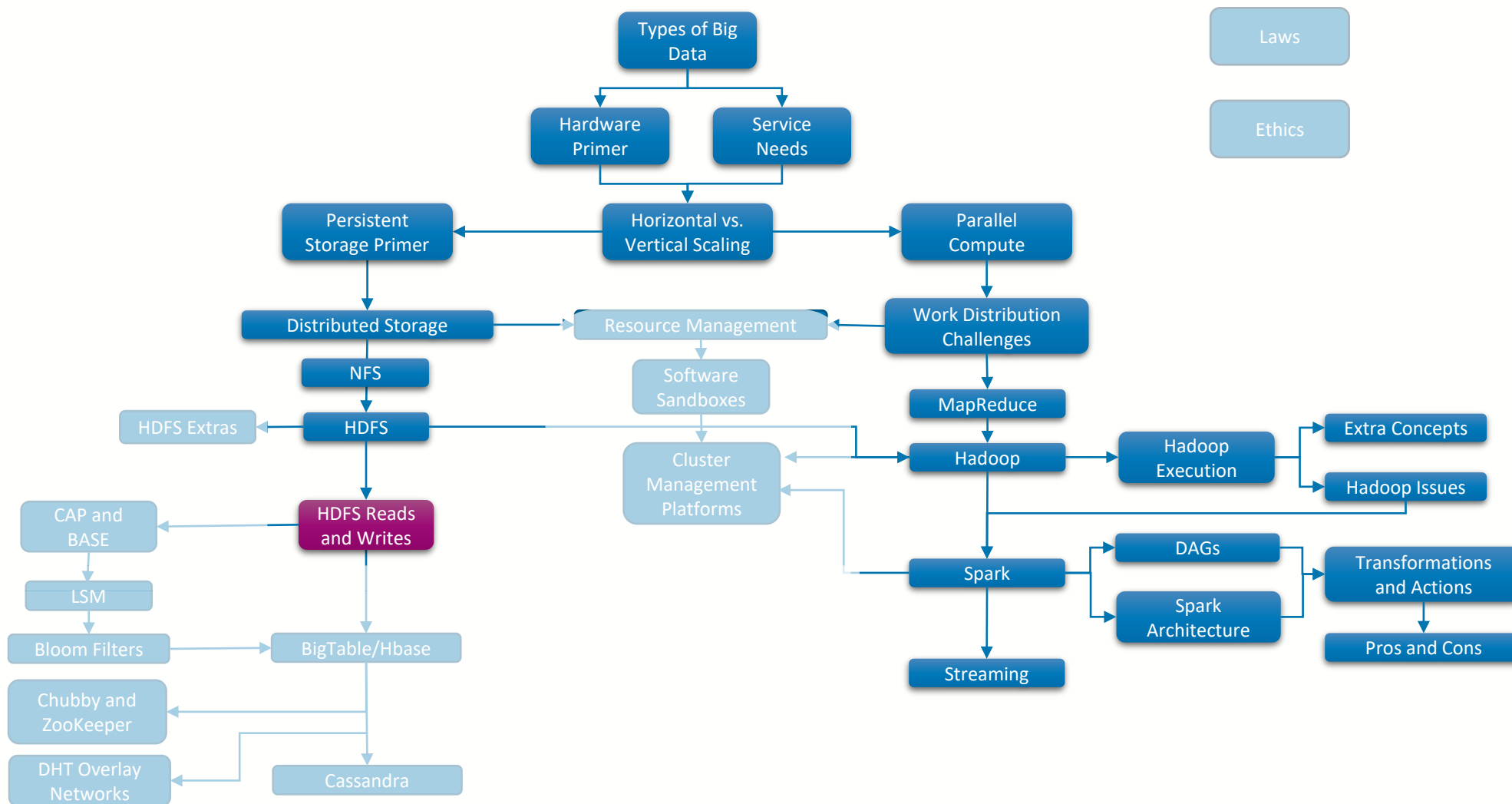
Dr. Richard McCreddie

**WORLD  
CHANGING  
GLASGOW**

**A WORLD  
TOP 100  
UNIVERSITY**



# Where Are We?





## Thinking about Trade-offs

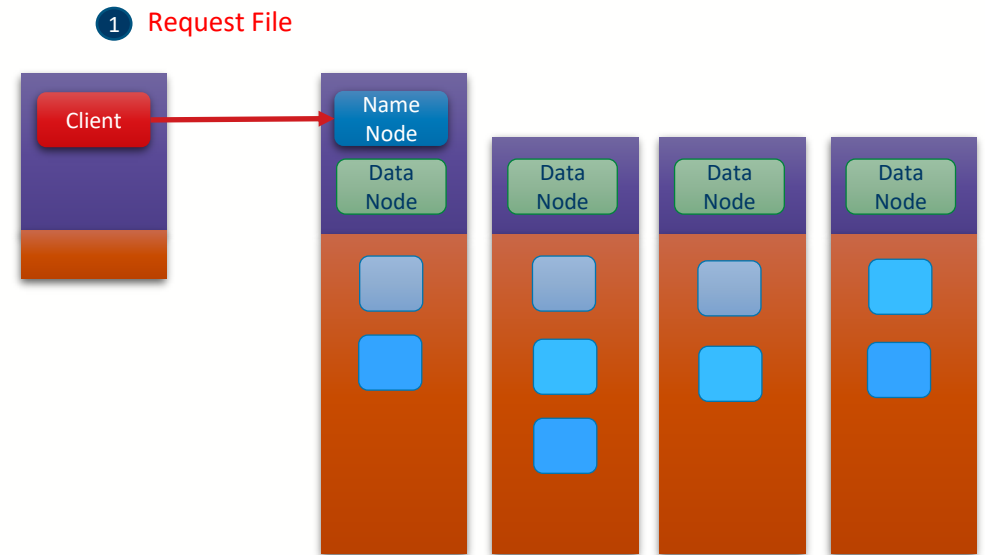
- The HDFS is a more complex distributed solution, and as such makes **trade-offs within its design**
- When considering one of these systems, a good way to think about one of these systems is to look at the **read** and **write** processes
  - How are they constructed?
  - What are they trying to avoid?





# HDFS Read Process

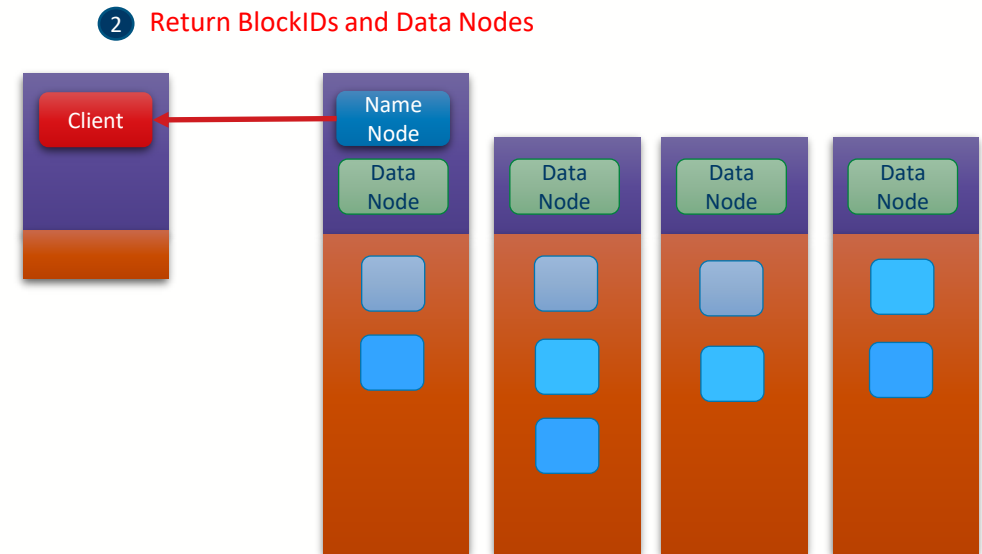
1. Client requests a file via the name node





# HDFS Read Process

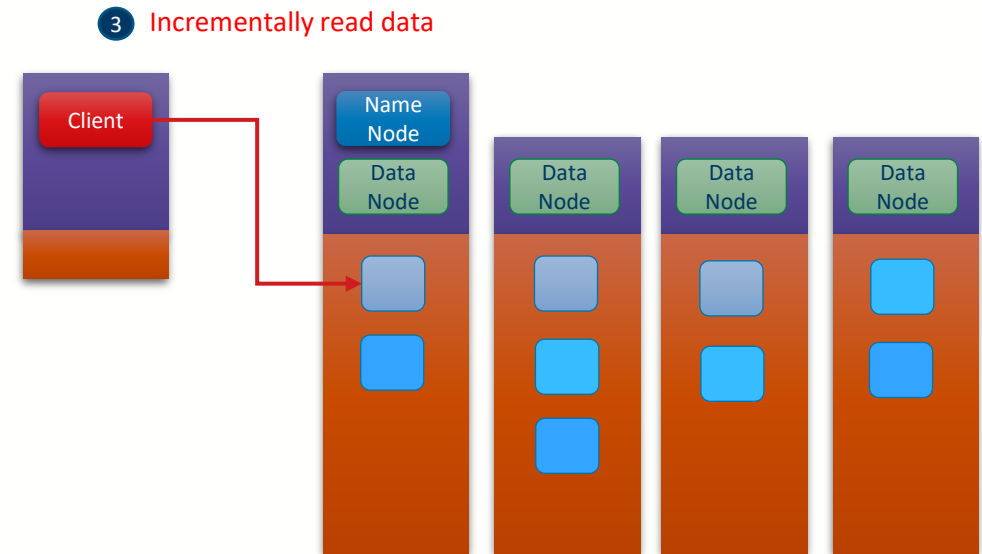
1. Client requests a file via the name node
2. Name Node returns
  - BlockIDs
  - Data Nodes holding those blocks





# HDFS Read Process

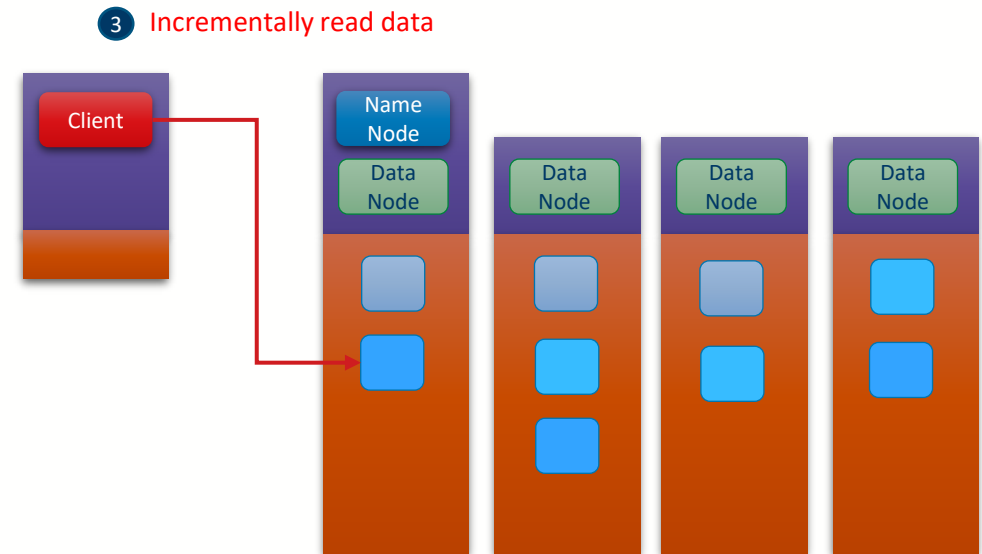
1. Client requests a file via the name node
2. Name Node returns
  - BlockIDs
  - Data Nodes holding those blocks
3. The Client decides which data nodes to access to get the blocks
  - Considers data locality





# HDFS Read Process

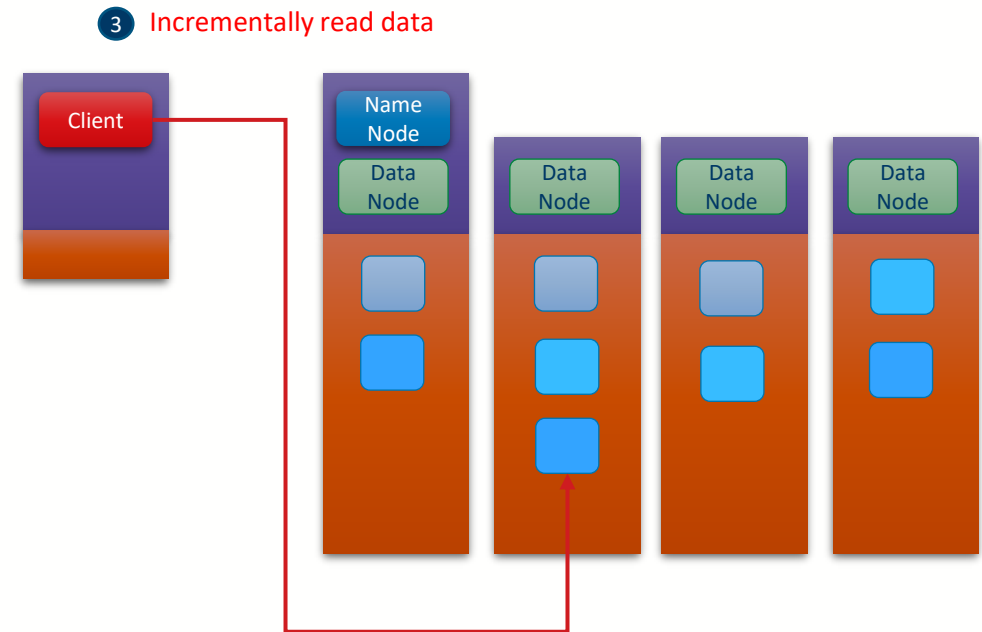
1. Client requests a file via the name node
2. Name Node returns
  - BlockIDs
  - Data Nodes holding those blocks
3. The Client decides which data nodes to access to get the blocks
  - Considers data locality





# HDFS Read Process

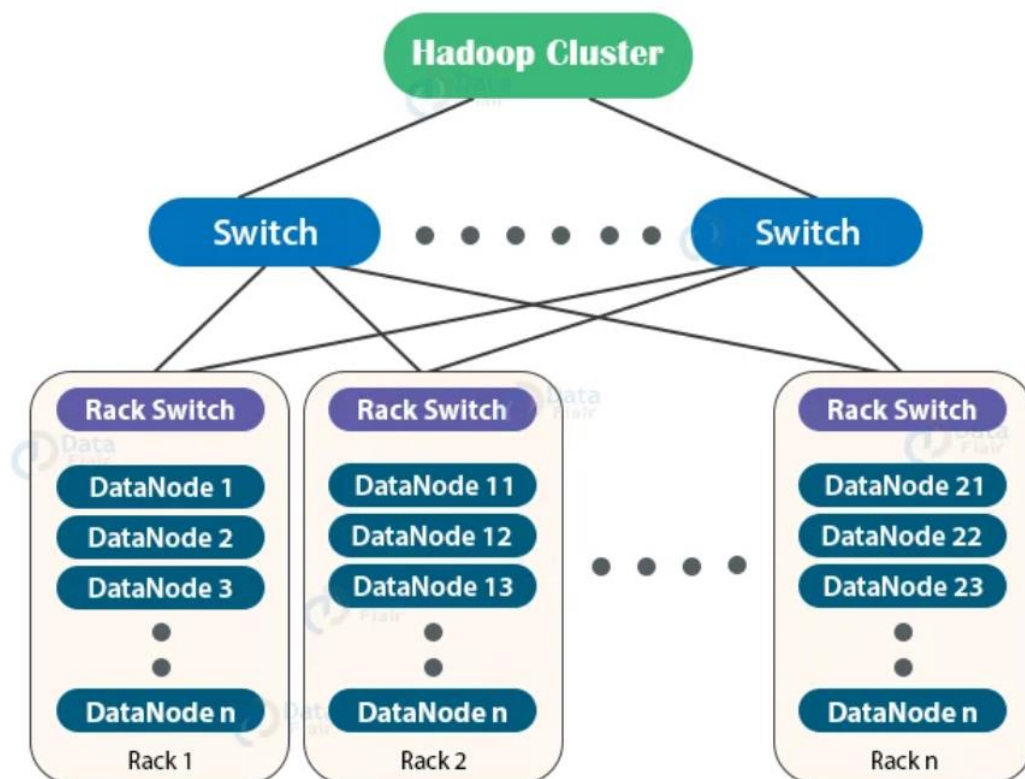
1. Client requests a file via the name node
2. Name Node returns
  - BlockIDs
  - Data Nodes holding those blocks
3. The Client decides which data nodes to access to get the blocks
  - Considers data locality





# What Design Decisions Went into this Process?

- **Name Node** (in earlier versions of the HDFS) is a singleton (there is only one), so can be overloaded
  - **Minimise client contact** with the Name Node (only do a single request)
- Data is **replicated in multiple places**, some may be closer (with regard to network latency) than others
  - **Select the closest one**, via Rack Awareness
    - Rack structure is manually configured in a file in the HDFS, not dynamically tracked



<https://data-flair.training/blogs/rack-awareness-hadoop-hdfs/>



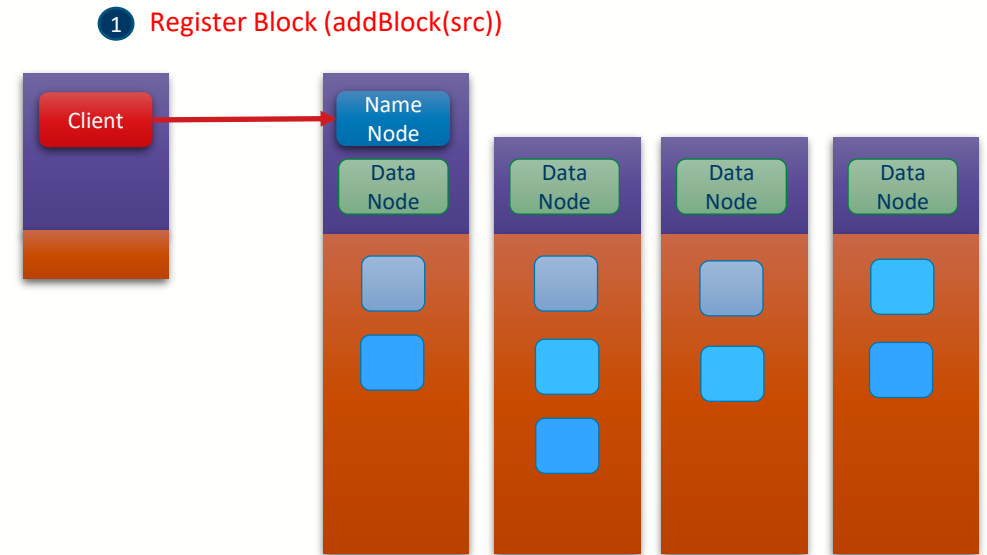
## What other options might there have been?

- When the Client decides which data node to write?
  - Contact all replicas, return when the first responds
    - Should have **fastest read time**, but **involves redundant requests** (expending disk read and network resources)
  - Contact a replica chosen uniformly at random so as to balance the load
    - Would have **slightly longer latency**, but might **work better when the cluster is heavily loaded** with requests
  - Contact the replica that is least loaded (and has the data)
    - **Like the above**, but **requires nodes to report their load** somehow (and which may change rapidly)
- As you can see, there was **choice made here by the HDFS designers**, where they wanted data locality, but wanted to minimise resource usage and avoid continually tracking how busy each data node is



# HDFS Write Process

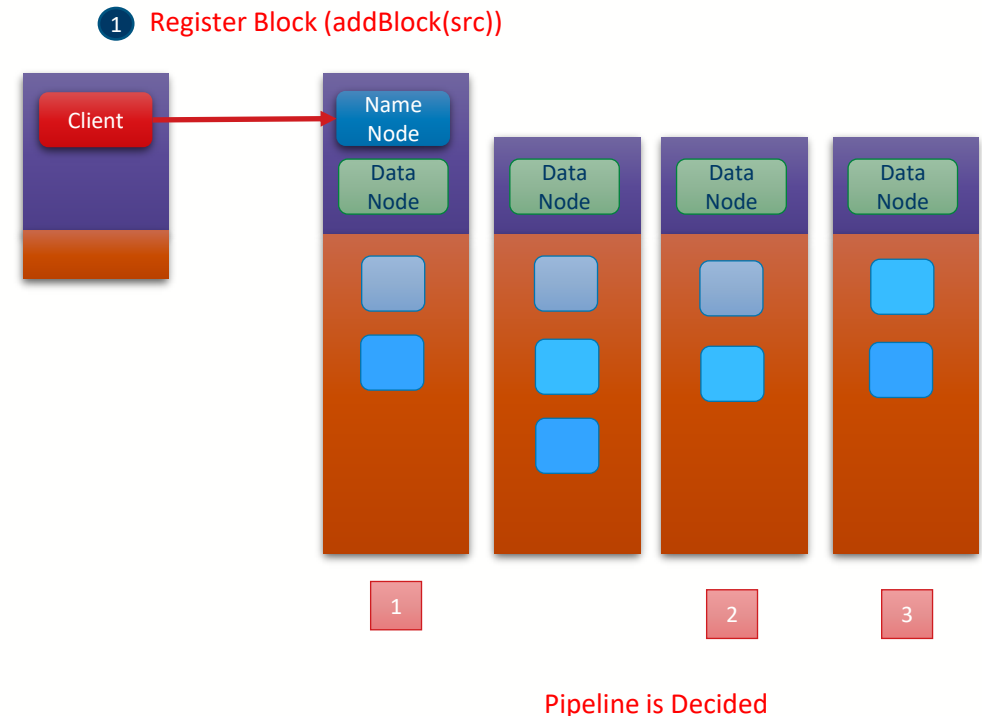
1. When a client writes, it first asks the NameNode to choose DataNodes to host replicas of the first block of the file.





# HDFS Write Process

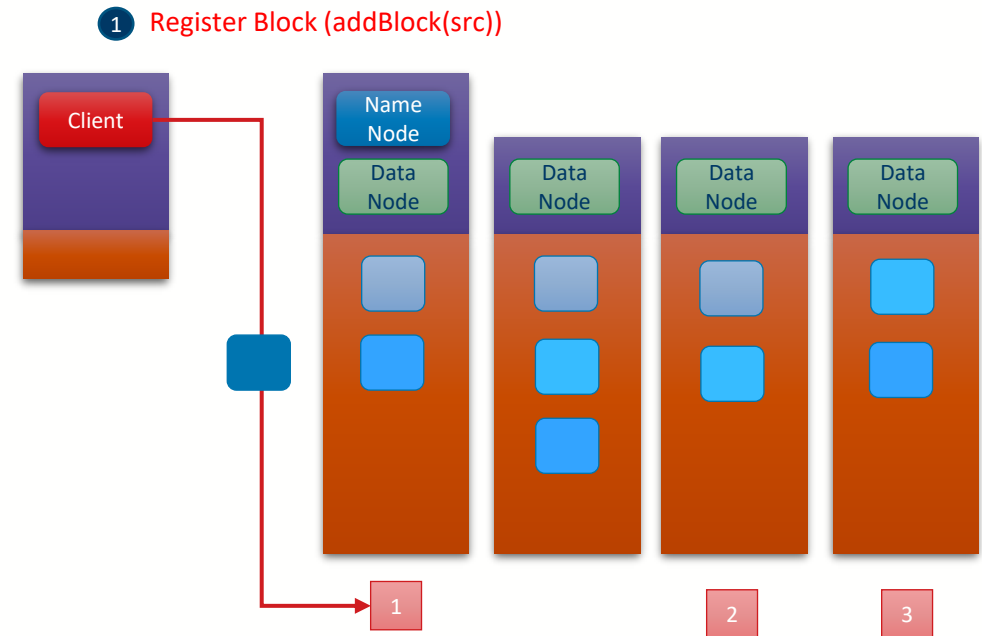
1. When a client writes, it first asks the NameNode to choose DataNodes to host replicas of the first block of the file.
  - The data nodes form a sequential pipeline that will hold the blocks





# HDFS Write Process

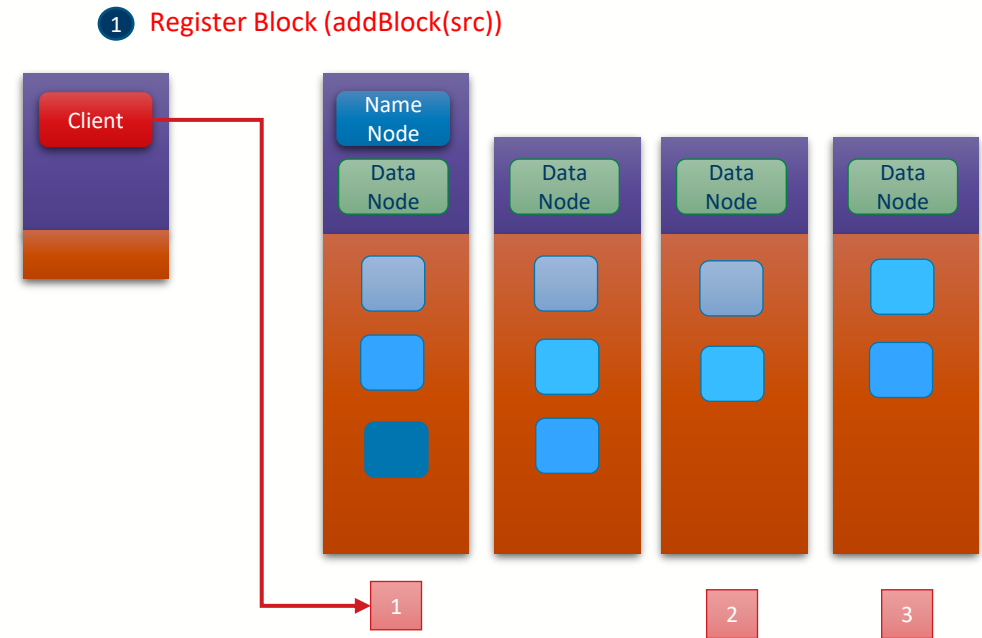
1. When a client writes, it first asks the NameNode to choose DataNodes to host replicas of the first block of the file.
  1. The data nodes form a sequential pipeline that will hold the blocks
2. The client organizes a pipeline from node-to-node and sends the data.





# HDFS Write Process

1. When a client writes, it first asks the NameNode to choose DataNodes to host replicas of the first block of the file.
  1. The data nodes form a sequential pipeline that will hold the blocks
2. The client organizes a pipeline from node-to-node and sends the data.

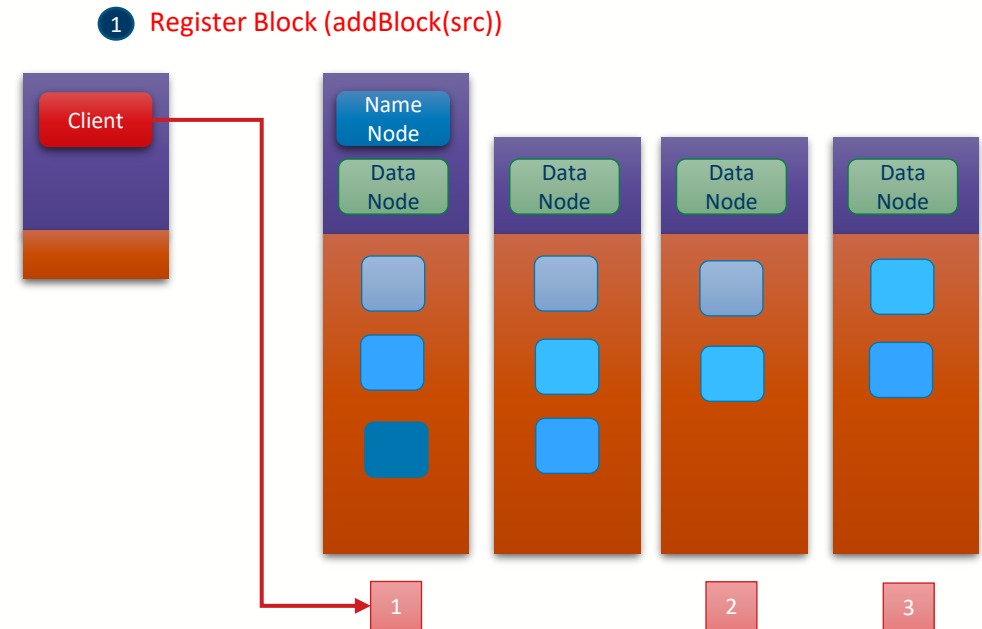






# HDFS Write Process

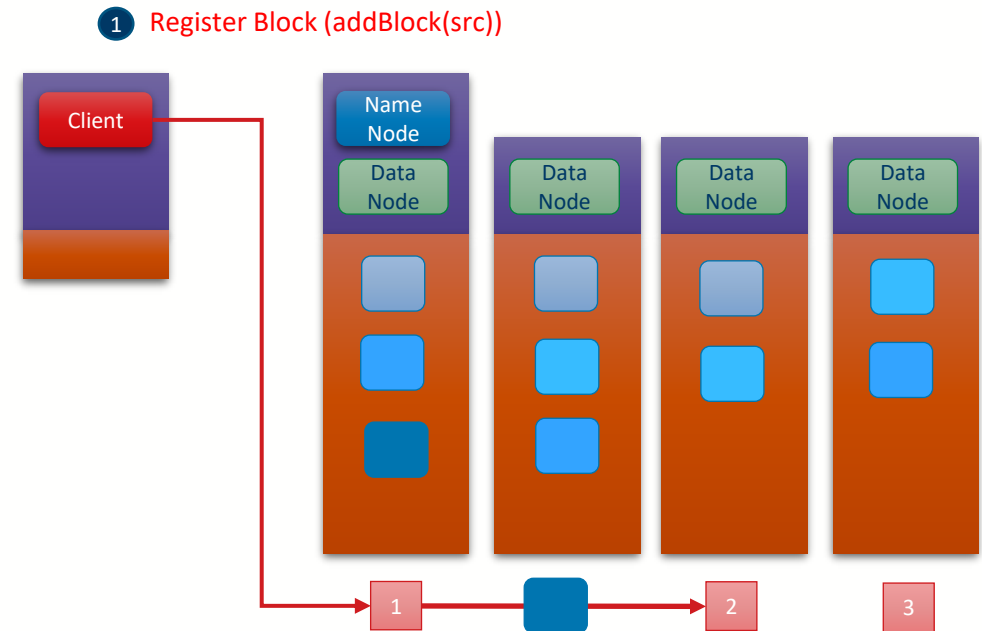
1. When a client writes, it first asks the NameNode to choose DataNodes to host replicas of the first block of the file.
  - The data nodes form a sequential pipeline that will hold the blocks
2. The client organizes a pipeline from node-to-node and sends the data.
  - Bytes are pushed to the pipeline as a sequence of packets. The bytes that an application writes first buffer at the client side.
  - After a packet buffer is filled (typically 64 KB), the data are pushed to the pipeline.





# HDFS Write Process

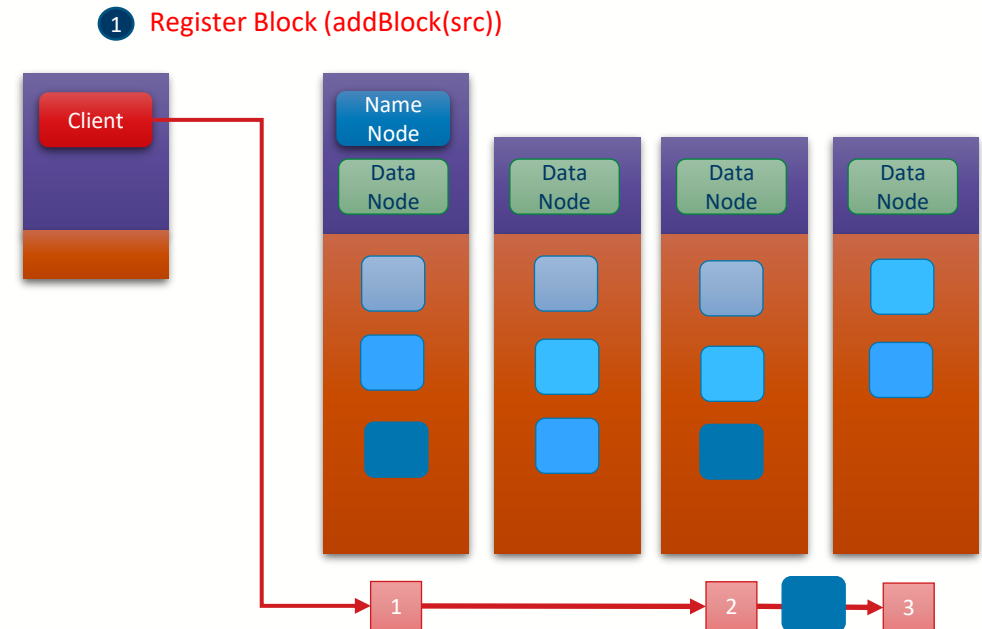
1. When a client writes, it first asks the NameNode to choose DataNodes to host replicas of the first block of the file.
  - The data nodes form a sequential pipeline that will hold the blocks
2. The client organizes a pipeline from node-to-node and sends the data.
  - Bytes are pushed to the pipeline as a sequence of packets. The bytes that an application writes first buffer at the client side.
  - After a packet buffer is filled (typically 64 KB), the data are pushed to the pipeline.
3. Writes occur in parallel within the pipeline, with data being forwarded from one node to the next





# HDFS Write Process

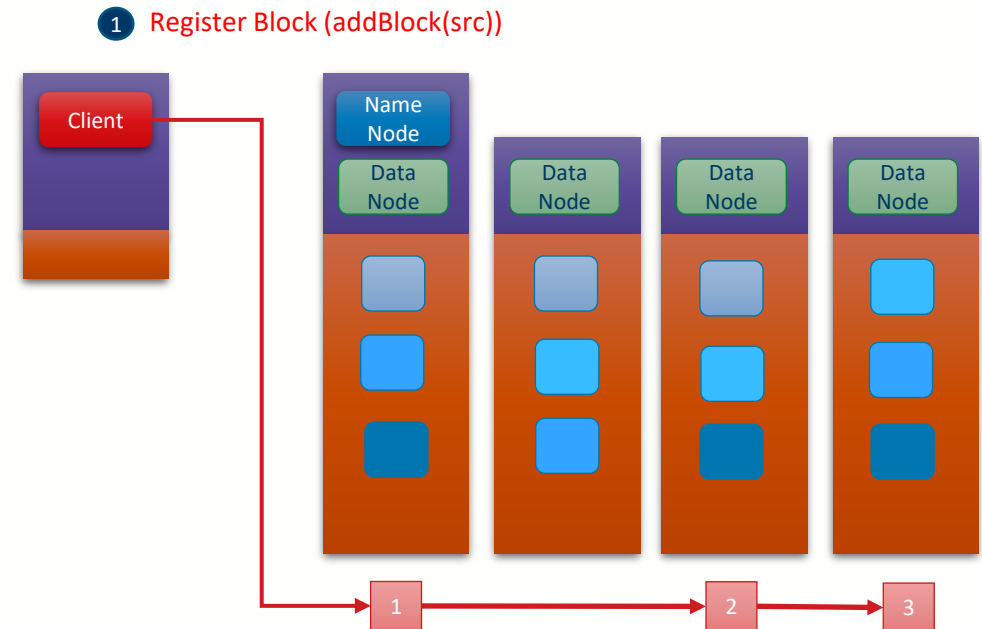
1. When a client writes, it first asks the NameNode to choose DataNodes to host replicas of the first block of the file.
  - The data nodes form a sequential pipeline that will hold the blocks
2. The client organizes a pipeline from node-to-node and sends the data.
  - Bytes are pushed to the pipeline as a sequence of packets. The bytes that an application writes first buffer at the client side.
  - After a packet buffer is filled (typically 64 KB), the data are pushed to the pipeline.
3. Writes occur in parallel within the pipeline, with data being forwarded from one node to the next





# HDFS Write Process

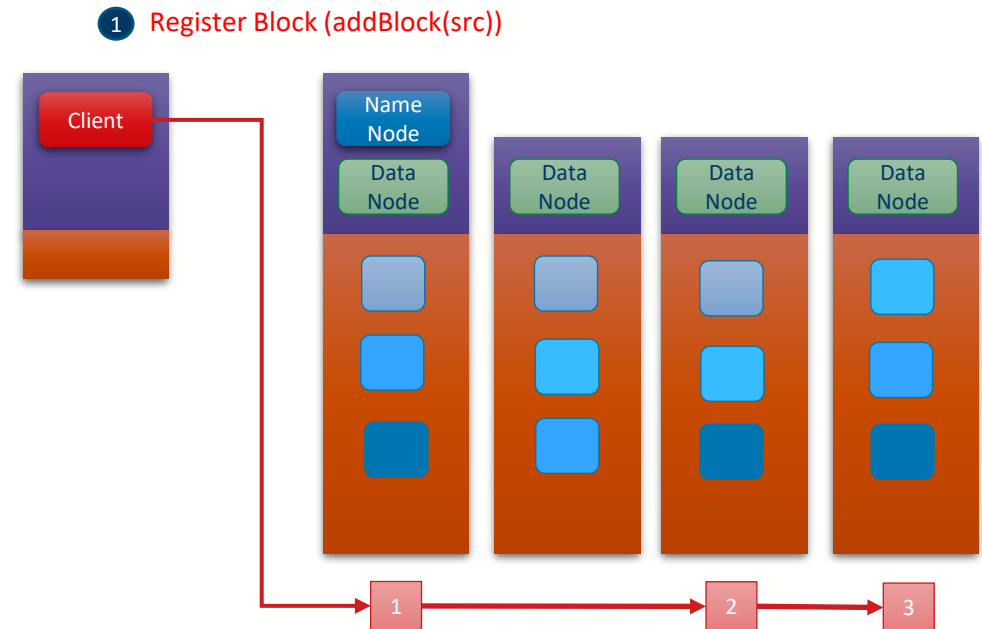
1. When a client writes, it first asks the NameNode to choose DataNodes to host replicas of the first block of the file.
  - The data nodes form a sequential pipeline that will hold the blocks
2. The client organizes a pipeline from node-to-node and sends the data.
  - Bytes are pushed to the pipeline as a sequence of packets. The bytes that an application writes first buffer at the client side.
  - After a packet buffer is filled (typically 64 KB), the data are pushed to the pipeline.
3. Writes occur in parallel within the pipeline, with data being forwarded from one node to the next





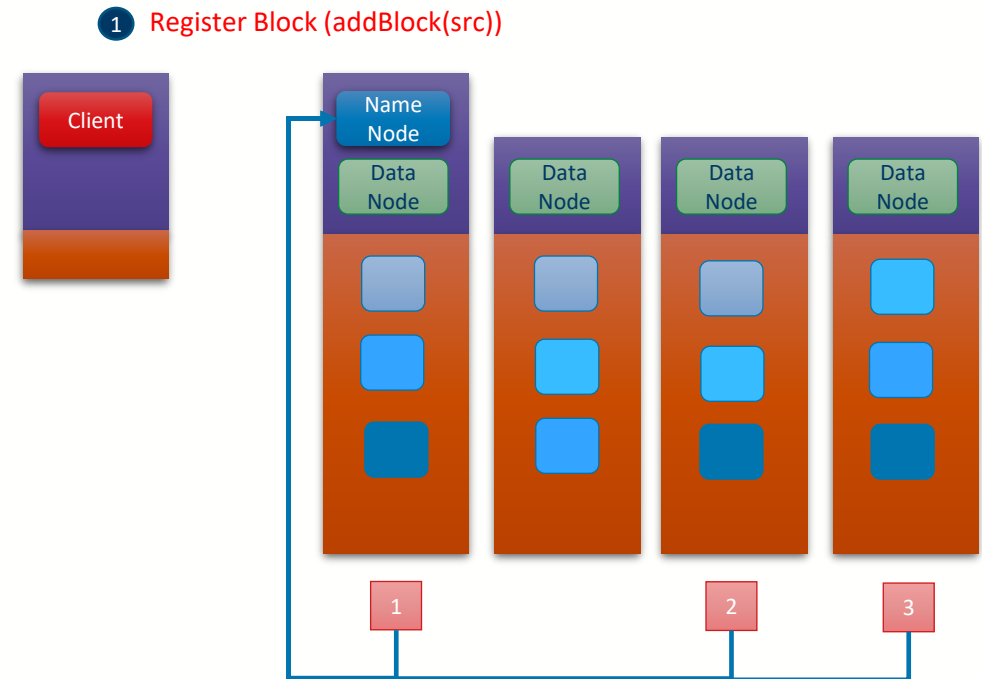
# HDFS Write Process

1. When a client writes, it first asks the NameNode to choose DataNodes to host replicas of the first block of the file.
  - The data nodes form a sequential pipeline that will hold the blocks
2. The client organizes a pipeline from node-to-node and sends the data.
  - Bytes are pushed to the pipeline as a sequence of packets. The bytes that an application writes first buffer at the client side.
  - After a packet buffer is filled (typically 64 KB), the data are pushed to the pipeline.
3. Writes occur in parallel within the pipeline, with data being forwarded from one node to the next
  - The next packet can be pushed to the pipeline before receiving the acknowledgement for the previous packets.
  - The number of outstanding packets is limited by the outstanding packets window size of the client.



# HDFS Write Process

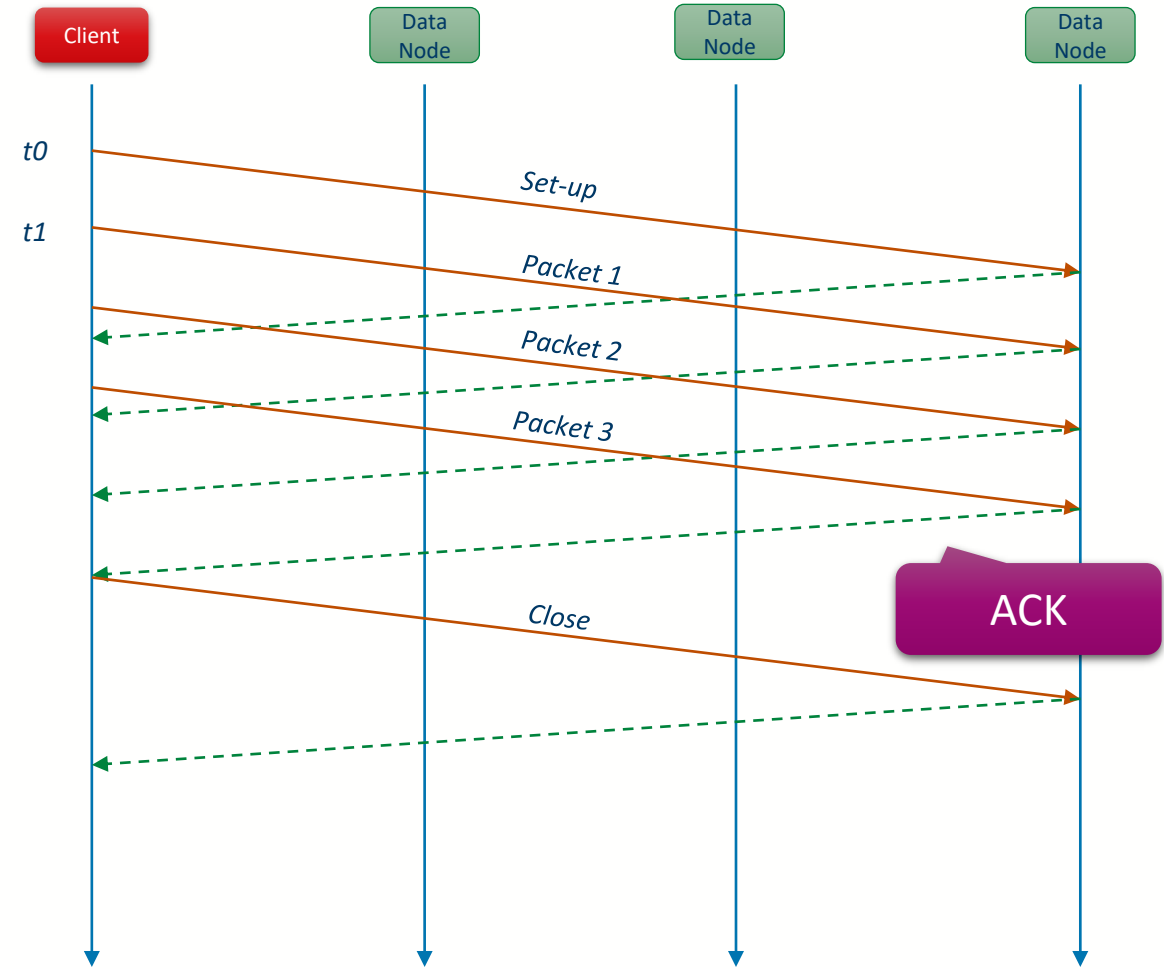
1. When a client writes, it first asks the NameNode to choose DataNodes to host replicas of the first block of the file.
  - The data nodes form a sequential pipeline that will hold the blocks
2. The client organizes a pipeline from node-to-node and sends the data.
  - Bytes are pushed to the pipeline as a sequence of packets. The bytes that an application writes first buffer at the client side.
  - After a packet buffer is filled (typically 64 KB), the data are pushed to the pipeline.
3. Writes occur in parallel within the pipeline, with data being forwarded from one node to the next
  - The next packet can be pushed to the pipeline before receiving the acknowledgement for the previous packets.
  - The number of outstanding packets is limited by the outstanding packets window size of the client.
4. Each DataNode reports the blocks that it receives to the NameNode





# Pipelining

- To be clear, multiple block packets can be 'in-flight' at the same time
  - Acknowledgements are returned when a packet is received
- hFlush will be periodically called to write packets to the journal, in case system failure occurs







## What Design Decisions Went into this Process?

- Again the **Name Node** is a bottleneck
  - Only **contact** it once
- We don't know **how fast or congested the network is**
  - Have the Client **send the data only once**
  - **Have each Data Node forward** what data it receives on to the next
- We want to **maximise transfer rates**
  - **Send packets in parallel**
  - Data Nodes can be **sending and receiving at the same time**
- We need to know **when the block is available**
  - **Have the Data Node report directly to the Name Node** once it has a whole block



## What other options might there have been?

- Client could have sent data to all data nodes simultaneously
  - Would have decreased the time until all nodes have the data, but might saturate the network connection on the client
- Client contacts all replicas one after the other
  - Spreads network traffic out over time, but takes longer
- Client sends data to first replica, then those forward to remaining replicas lazily (i.e., when idle)
  - Avoids overloading individual data nodes, but may take a long time before full replication is achieved



University  
of Glasgow

Course Coordinator  
**Dr. Richard McCreadie**

**Email:** richard.mccreadie@glasgow.ac.uk  
**Room:** SAWB 304

**#UofGWorldChangers**



**@UofGlasgow**